

How the Text Dataset Is Prepared

A provenance report for the tea pest and disease corpus

Summary

The text side of this project is **generated, not collected**. No field note was written by a farmer or an agronomist. Every note is composed by a program from hand-authored phrase pools, conditioned only on the class label that the image annotation already carries.

Two separate text corpora exist, produced by two different generators and used for two different purposes. They must not be confused:

	Corpus A — crop-linked notes	Corpus B — symptom bundle
File	tea_results/annotation/annotations.csv	data_final/text_data/annotations.csv
Generator	backend/tea_annotator.py	generate_tea_disease_text()
Rows	371 (one per image crop)	3,000
Unique	323	673
Linked to an image?	Yes, by exact box key	No
Used for	the diagnostic model	the advisory retrieval evaluation

1 Corpus A: the crop-linked field notes

This is the corpus behind every diagnostic number in the paper. Figure ?? shows the pipeline. The image branch and the text branch are produced independently and then joined on a key that the crop step creates.

IMAGE BRANCH

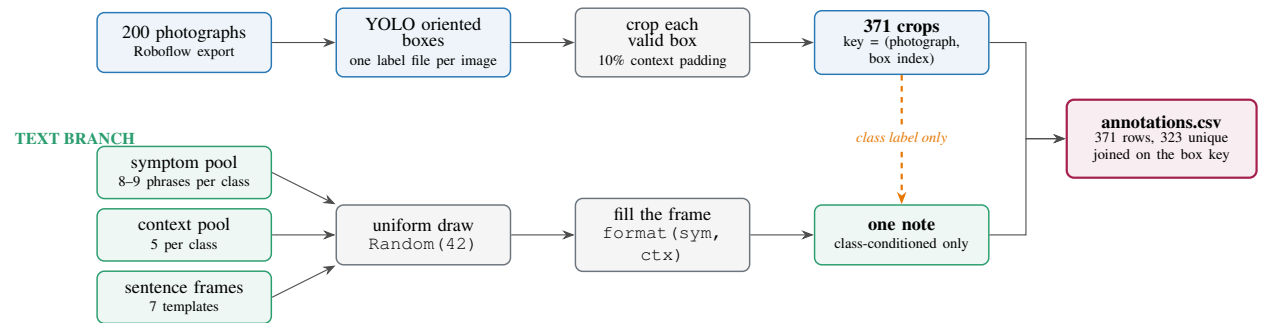


Figure 1: Preparation of the crop-linked corpus. The image branch produces the pairing key; the text branch never sees the photograph. The dashed arrow is the only information that crosses between them — the class label — which is why a note describes a *class* rather than the specific leaf it is attached to.

1.1 What the generator actually does

backend/tea_annotator.py holds three hand-authored pools per class and composes one note per crop:

```
sym = rng.choice(_SYMPTOMS[disease]); ctx = rng.choice(_CONTEXT[disease]);
tmpl = rng.choice(_TEMPLATES_PLAIN); return tmpl.format(symptom=sym,
context=ctx)
```

The draw is seeded with `random.Random(42)`, so the corpus is reproducible. Pool sizes and the resulting combinatorial space are:

Class	Symptoms	Contexts	Frames	Distinct notes possible	Crops
LOOPER_CATERPILLARS	9	5	7	315	102
LEAF_BLIGHT	8	5	7	280	129
LEAF_HOPPERS	8	5	7	280	9
LEAF_RUST	8	5	7	280	67
MOSQUITO_BUG	8	5	7	280	64
Total	41	25	7	1,435	371

371 notes are drawn from 1,435 possible strings, so collisions occur: the realised corpus contains **323 unique texts**, i.e. 48 notes are exact duplicates of another note. A representative row reads:

*Crop health log: Visible symptom — irregular brown lesions spreading along leaf veins with dark borders.
Observation context: high humidity and warm temperatures favoured disease spread.*

which is frame 6 filled with symptom 2 and a matching context string.

1.2 The unused image-grounded mode

The same script has an `--auto_caption` mode that runs `Salesforce/blip-image-captioning-base` over each crop and splices the real caption into five alternative frames. **The committed corpus does not use it:** every stored note matches the plain frames, so no image-derived content is present in the text. This matters — under the BLIP mode the text would carry genuine per-leaf visual evidence, and the cross-modal results would mean something different.

2 Corpus B: the bundled symptom corpus

The advisory retrieval evaluation uses a second, independently generated corpus. Its base pools live in `generate_tea_disease_text()`, which holds six sentences per class and appends, with probability 0.35, one of four deliberately ambiguous suffixes such as “*Pattern may overlap with adjacent stress factors.*” Its stated design goal is the opposite of Corpus A:

“Uses purely visual symptom descriptions — no scientific genus names, no disease labels embedded in the text. This prevents trivial keyword-matching shortcuts.”

Measured directly from the delivered file, the corpus has the following shape:

Property	Value
Rows delivered	3,000 (600 per class)
Distinct observations	673
Exact duplicate rows	2,327
Distinct sentences in the whole corpus	131
Sentences appearing under more than one class	43
Sentences per observation	2–6 (mean 4.09)
Observations with no class-unique sentence	182 (27.0%)

Observations are therefore assembled from a small shared sentence inventory, and a third of that inventory is not class-specific. This is the direct cause of the two-regime behaviour reported in the advisory evaluation: retrieval reaches 99.24% precision@1 on queries carrying at least one class-unique sentence, and falls to 72.46% on queries built only from shared sentences.

3 What this means for the results

Three consequences follow, and each is stated in the paper rather than left implicit.

The text-only score is not a language-understanding result. Because a note is generated *from* the label, text-only classification is close to reading the answer back. This is why text-only reaches 96.00% accuracy while image-only reaches 61.33% on the same 75 crops. A 178-token vocabulary of target-revealing words is fitted on training annotations only and masked everywhere, which removes explicit disease names, but class-specific symptom phrasing necessarily remains.

“Exact pairing” means exact bookkeeping, not co-observation. The (photograph, box index) key guarantees that a crop and its note never land in different splits, which is a real leakage control and an improvement over the earlier label-aligned proxy pairing. It does not make the note an observation *of that leaf*. The evidence is in the retrieval numbers: class-level text-to-image recall@1 is 0.6000 and image-to-text is 0.6133, while **exact** text-to-image recall@1 is 1.33%. The joint space is class-compatible, not instance-level.

The advisory numbers are upper bounds. Corpus B’s queries and its retrieval base are disjoint observations but share sentence vocabulary, so 90.05% precision@1 should be read as an upper bound relative to free-form notes.

Reproducing the corpora

```
python backend/tea_annotator.py                (Corpus A, template mode, seed 42)
python backend/tea_annotator.py --auto_caption (image-grounded mode, unused)
python backend/tea_annotator.py --save_crops   (also writes the 371 crop JPEGs)
```

Corpus B ships pre-generated inside the data bundle. Its base sentence pools live in

```
backend/FarmFederateColabComplete.py
```

The exact composer that assembled the delivered multi-sentence observations was not located in the current tree, so the shape reported in Section 2 is measured from the delivered file rather than re-derived from source.